

Zadanie 1 (30 min) — aplikacja produkty, BEZ `INotifyPropertyChanged`

1. Zrób MAUI app „Lista produktów”.
 2. Klasa `Produkt`: `Nazwa` (`string`) i `Ilosc` (`int`) jako zwykłe `{ get; set; }` (bez `INotifyPropertyChanged`).
 3. `ObservableCollection<Produkt> Produkty` + 3 startowe wpisy (np. Mleko 3, Chleb 5, Jabłko 12).
 4. XAML: `CollectionView` (`ItemsSource=Produkty`, `SelectionMode=Single`) pokazuje `Nazwa` i „Ilość: X”.
 5. Pod listą `Label`: „Wybrałeś: {SelectedItem.Nazwa}”.
 6. Edycja wybranego: `Entry` dla `SelectedItem.Nazwa` (`TwoWay`), `Entry` dla `SelectedItem.Ilosc` (`TwoWay`), `Stepper` 0–50 dla `SelectedItem.Ilosc` (`TwoWay`) + `Label` z ilością.
 7. Test: zmieniaj nazwę/ilość i sprawdź, czy element w liście zawsze odświeża się od razu.
-

Zadanie 2 (30 min) — aplikacja produkty, Z `INotifyPropertyChanged`

1. Skopiuj zadanie 1.
2. W klasie `Produkt` dodaj `INotifyPropertyChanged` i wywołuj `PropertyChanged` w setterach `Nazwa` i `Ilosc`.
3. Dodaj `DataTrigger` w elemencie listy (`Frame`): jeśli `Ilosc == 0`, ustaw tło na `LightPink` (brak na stanie).
4. Test: zmiany z `Entry/Stepper` mają natychmiast aktualizować listę + kolor przy `Ilosc=0`.
5. Dopisz 1 zdanie: „Po co jest `PropertyChanged` w tej aplikacji?”.